Lab 26 — Case-Based Reasoning (Simple CBR System)

Aim

To implement a simple **Case-Based Reasoning (CBR)** system that retrieves the most similar past case and reuses its solution for a new problem.

Algorithm / Procedure

1. Create a case library containing previous cases and their solutions.
2. Represent each case using a set of features.
3. Compare the new case with each stored case.
4. Calculate similarity based on the number of matching features.
5. Retrieve the case with the highest similarity.
6. Reuse the solution of the retrieved case.
7. Retain the new case in the case library for future use.
8. Display the retrieved solution and updated library size.

Program

```
case_library=[{"symptoms":{"fever":True,"cough":True,"fatigue":False},"diagnosis":"Flu"},
{"symptoms":{"fever":False,"cough":True,"fatigue":True},"diagnosis":"Cold"},{"symptoms
":{"fever":True,"cough":False,"fatigue":True},"diagnosis":"Viral Infection"}]

def similarity(a,b):

    return sum(1 for k in a if a[k]==b.get(k))

def retrieve(new_case):
```

```
    scored=[(similarity(new_case,c["symptoms"]),c) for c in case_library]
    return max(scored,key=lambda x:x[0])[1]
new_symptoms={"fever":True,"cough":True,"fatigue":False}
best_case=retrieve(new_symptoms)
print("Retrieved diagnosis (Reuse):",best_case["diagnosis"])
case_library.append({"symptoms":new_symptoms,"diagnosis":best_case["diagnosis"]})
print("Case library size after Retain:",len(case_library))
```

Output

Retrieved diagnosis (Reuse): Flu

Case library size after Retain: 4

Result

The Case-Based Reasoning system was successfully implemented. It retrieved the most similar previous case, reused its diagnosis, and retained the new case in the case library for future use.